

Kubernetes Interview Guide

Top 20 Most-Asked Questions & Answers

With Practical Examples & Real-World Scenarios

20

Questions
∞

Examples
100%

Topics Covered

- ✓ Core Concepts
- ✓ Networking & Services
- ✓ Scaling & Probes
- ✓ Troubleshooting
- ✓ Workloads & Controllers
- ✓ Configuration & Secrets
- ✓ Security & RBAC
- ✓ Control Plane Components

Table of Contents

Q0

1 What is Kubernetes?

Q0

2 What is a Pod in Kubernetes?

Q0

3 Difference between Deployment and StatefulSet?

Q0

4 What is CrashLoopBackOff?

Q0

5 What is the difference between Requests and Limits?

Q0

6 What is a Service in Kubernetes?

Q0

7 Difference between NodePort, LoadBalancer, and Ingress?

Q0

8 What is ConfigMap?

Q0

9 What are Secrets in Kubernetes?

Q1

0 What is HPA (Horizontal Pod Autoscaler)?

Q1

1 What is OOMKilled?

Q1

2 Difference between Liveness and Readiness Probe?

Q1

3 What is etcd?

Q1

4 What happens when you run `kubectl apply`?

Q1

5 What is kube-proxy?

Q1

6 What is a Namespace?

Q1

7 What causes Pods to remain in Pending state?

Q1

8 What is a DaemonSet?

Q1

9 What is a ReplicaSet?

Q2

0 What is RBAC?

Q0
1

What is Kubernetes?

Kubernetes (K8s) is an open-source **container orchestration platform** originally developed by Google and now maintained by the CNCF. It automates the deployment, scaling, and management of containerised applications across a cluster of machines.

Key Capabilities:

- **Auto-scheduling** – places containers on the right nodes based on resource needs
- **Self-healing** – restarts failed containers, replaces dead nodes
- **Horizontal scaling** – scale up/down automatically or manually
- **Service discovery & load balancing** – built-in DNS and traffic routing
- **Rolling updates & rollbacks** – zero-downtime deployments

Example:

```
# Check the cluster info
kubectl cluster-info

# List all nodes in the cluster
kubectl get nodes

# Get cluster version
kubectl version --short
```

Think of Kubernetes as an *operating system for your data centre* — it abstracts away the underlying infrastructure so you focus on applications.

What is a Pod in Kubernetes?

A **Pod** is the smallest and most basic deployable unit in Kubernetes. It encapsulates one or more containers that share the same network namespace (IP address, ports) and storage volumes.

Important Facts:

- All containers in a Pod communicate via **localhost**
- Pods are **ephemeral** – they can be killed and recreated anytime
- Each Pod gets its own **unique IP** within the cluster
- Multi-container Pods use the **sidecar**, **ambassador**, or **adapter** patterns

Example YAML:

```
apiVersion: v1
kind: Pod
metadata:
  name: my-app-pod
  labels:
    app: my-app
spec:
  containers:
    - name: web
      image: nginx:1.25
      ports:
        - containerPort: 80
    - name: log-sidecar          # sidecar container
      image: busybox
      command: ["sh", "-c", "tail -f /var/log/nginx/access.log"]
```

Difference between Deployment and StatefulSet?

Both manage Pod replicas, but they serve different application types.

Feature	Deployment	StatefulSet
Identity	Random Pod names	Stable, ordered names (pod-0, pod-1)
Storage	Shared / no persistence	Each Pod gets its own PVC
Scaling	Parallel, any order	Ordered, one at a time
Use case	Stateless apps (web, API)	Stateful apps (DB, Kafka, Zookeeper)
DNS	Service DNS only	Stable per-Pod DNS headless service

StatefulSet Example (MySQL):

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: mysql
spec:
  serviceName: "mysql"
  replicas: 3
  selector:
    matchLabels: {app: mysql}
  template:
    metadata:
      labels: {app: mysql}
    spec:
      containers:
        - name: mysql
          image: mysql:8.0
          volumeMounts:
            - name: data
              mountPath: /var/lib/mysql
  volumeClaimTemplates:
    - metadata:
        name: data
      spec:
        accessModes: ["ReadWriteOnce"]
        resources:
          requests:
            storage: 10Gi
```

What is CrashLoopBackOff?

CrashLoopBackOff means a container is repeatedly crashing after startup. Kubernetes tries to restart it, but each restart is delayed exponentially (10s → 20s → 40s → ... → 5min) to avoid thrashing.

Common Causes:

- Application error / unhandled exception at startup
- Missing environment variables or config
- Wrong entrypoint command or missing binary
- Dependency not ready (e.g. DB not up yet)
- OOM (Out Of Memory) kill

Debugging Commands:

```
# Check the Pod status
kubectl get pods

# Read the logs of the crashing container
kubectl logs my-pod --previous

# Describe to see events and last state
kubectl describe pod my-pod

# Exec into the container (if it briefly starts)
kubectl exec -it my-pod -- /bin/sh
```

What is the difference between Requests and Limits?

Resource **Requests** and **Limits** control how much CPU and memory a container can use.

- **Request** – the *minimum* guaranteed resource. Used by the scheduler to decide which node to place the Pod on.
- **Limit** – the *maximum* resource the container may consume. Exceeding the CPU limit causes throttling; exceeding the memory limit causes OOMKill.

Example:

```
resources:
  requests:
    cpu: "250m"      # 0.25 CPU cores guaranteed
    memory: "256Mi"  # 256 MiB guaranteed
  limits:
    cpu: "1"         # Max 1 CPU core (throttled if exceeded)
    memory: "512Mi"  # Max 512 MiB (OOMKilled if exceeded)
```

QoS Classes (based on requests/limits):

- **Guaranteed** – requests == limits for all containers
- **Burstable** – requests < limits
- **BestEffort** – no requests or limits set (first to be evicted)

What is a Service in Kubernetes?

A **Service** is a stable, virtual endpoint that provides a consistent IP address and DNS name to access a dynamic set of Pods. Because Pods are ephemeral and their IPs change, Services provide a reliable abstraction layer.

Service Types:

- **ClusterIP** (default) – internal-only; accessible only within the cluster
- **NodePort** – exposes the Service on each node's IP at a static port (30000–32767)
- **LoadBalancer** – provisions a cloud load balancer (AWS ELB, GCP LB, etc.)
- **ExternalName** – maps to an external DNS name

Example:

```
apiVersion: v1
kind: Service
metadata:
  name: my-svc
spec:
  selector:
    app: my-app      # targets Pods with this label
  ports:
    - port: 80        # Service port
      targetPort: 8080 # Pod port
  type: ClusterIP
```

Difference between NodePort, LoadBalancer, and Ingress?

Feature	NodePort	LoadBalancer	Ingress
Layer	L4 (TCP/UDP)	L4 (TCP/UDP)	L7 (HTTP/HTTPS)
External IP	Node IP	Cloud LB IP	Ingress controller IP
Path routing	No	No	Yes (host/path rules)
TLS/SSL	No	No (at LB level)	Yes (cert termination)
Cost	Free	Cloud LB cost	One LB + routing
Best for	Dev/testing	Single Service	Multi-service production

Ingress Example (NGINX):

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: my-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  rules:
  - host: api.example.com
    http:
      paths:
      - path: /v1
        pathType: Prefix
        backend:
          service:
            name: api-v1-svc
            port: {number: 80}
      - path: /v2
        pathType: Prefix
        backend:
          service:
            name: api-v2-svc
            port: {number: 80}
```

What is ConfigMap?

A **ConfigMap** stores non-sensitive configuration data as key-value pairs. It decouples configuration from the container image, making apps portable across environments (dev, staging, prod).

Ways to use a ConfigMap:

- As **environment variables**
- As a **mounted file/volume**
- As **command-line arguments**

Example:

```
# Create a ConfigMap
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  DB_HOST: "postgres.default.svc"
  LOG_LEVEL: "info"
  config.yaml: |
    server:
      port: 8080
      timeout: 30s

---
# Use it in a Pod
spec:
  containers:
    - name: app
      image: my-app
      envFrom:
        - configMapRef:
            name: app-config
      volumeMounts:
        - name: config-vol
          mountPath: /etc/config
  volumes:
    - name: config-vol
      configMap:
        name: app-config
```

What are Secrets in Kubernetes?

A **Secret** stores sensitive data such as passwords, tokens, and TLS certificates. Unlike ConfigMaps, Secrets are base64-encoded and can be encrypted at rest using an EncryptionConfiguration.

Secret Types:

- **Opaque** – generic key-value (default)
- **kubernetes.io/tls** – TLS certificate and key
- **kubernetes.io/dockerconfigjson** – Docker registry credentials
- **kubernetes.io/service-account-token** – SA tokens

Example:

```
# Create a secret imperatively
kubectl create secret generic db-creds \
  --from-literal=username=admin \
  --from-literal=password=s3cr3t!

# YAML form (values are base64-encoded)
apiVersion: v1
kind: Secret
metadata:
  name: db-creds
type: Opaque
data:
  username: YWRtaW4=      # echo -n 'admin' | base64
  password: czNjcjN0IQ==

# Consume as env variable
env:
- name: DB_PASSWORD
  valueFrom:
    secretKeyRef:
      name: db-creds
      key: password
```

Note: base64 is NOT encryption. Always enable EncryptionConfiguration and use RBAC to restrict Secret access.

Q1
0

What is HPA (Horizontal Pod Autoscaler)?

The **Horizontal Pod Autoscaler (HPA)** automatically scales the number of Pod replicas in a Deployment or ReplicaSet based on observed CPU/memory utilisation or custom metrics.

How it works:

- HPA queries the **Metrics Server** every 15 seconds
- Compares observed metric vs target threshold
- Calculates desired replicas: $desired = \text{ceil}(current \times (current_metric / target_metric))$
- Scales within configured min/max bounds

Example:

```
# Imperative
kubectl autoscale deployment my-app \
  --cpu-percent=50 --min=2 --max=10

# Declarative YAML
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: my-app-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: my-app
  minReplicas: 2
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 50
```

Q1
1

What is OOMKilled?

OOMKilled (Out Of Memory Killed) means the Linux kernel terminated a container because it exceeded its memory limit. The container exit code is typically **137** (128 + SIGKILL).

How to detect it:

```
kubectl describe pod my-pod
# Look for:
# Last State: Terminated
#   Reason: OOMKilled
#   Exit Code: 137
```

How to fix it:

- Increase the memory **limit** in the Pod spec
- Profile your application to find memory leaks
- Use a Java app? Set -XX:MaxRAMPercentage instead of a fixed heap
- Enable **VPA** (Vertical Pod Autoscaler) for automatic right-sizing

Prevention Example:

```
resources:
  requests:
    memory: "256Mi"
  limits:
    memory: "512Mi"    # Raise this if you see OOMKilled
# Java JVM option to respect container limits:
# env  JAVA_OPTS="-XX:MaxRAMPercentage=75.0"
```

Q1
2

Difference between Liveness and Readiness Probe?

Aspect	Liveness Probe	Readiness Probe
Purpose	Is the app alive?	Is the app ready to serve traffic?
On failure	Restart the container	Remove Pod from Service endpoints
Use when	App can deadlock without crashing	App needs time to warm up / depends on external
Frequency	Continuously	Continuously

Example with both probes:

```
livenessProbe:
  httpGet:
    path: /healthz
    port: 8080
  initialDelaySeconds: 15  # wait before first check
  periodSeconds: 20
  failureThreshold: 3

readinessProbe:
  httpGet:
    path: /ready
    port: 8080
  initialDelaySeconds: 5
  periodSeconds: 10
  failureThreshold: 3
```

*Tip: A third probe, **startupProbe**, protects slow-starting containers by disabling liveness/readiness until the app has fully started.*

Q1
3

What is etcd?

etcd is a distributed, consistent key-value store that serves as Kubernetes' **source of truth**. Every cluster object — Pods, Services, ConfigMaps, Secrets, RBAC policies — is persisted in etcd.

Key Characteristics:

- Uses the **Raft consensus algorithm** for strong consistency
- Typically deployed with **3 or 5 nodes** for HA (requires quorum: $n/2 + 1$)
- Only the **API server** talks to etcd directly
- Data is stored at `/registry/<resource-type>/<namespace>/<name>`

Useful etcd Commands:

```
# Check etcd health (inside a control-plane node)
ETCDCTL_API=3 etcdctl \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key \
  endpoint health

# List all keys
etcdctl get / --prefix --keys-only
```

Q1
4

What happens when you run `kubectl apply`?

Running `kubectl apply -f manifest.yaml` triggers a multi-step workflow inside the Kubernetes control plane:

1. **kubectl** sends an HTTP PATCH/PUT to the API server with the desired state
2. **API Server** authenticates (AuthN) and authorises (AuthZ via RBAC) the request
3. **Admission Controllers** validate and potentially mutate the object (e.g. inject sidecar)
4. **API Server** writes the object to **etcd**
5. **Controller Manager** detects the change via a Watch and reconciles actual → desired state (e.g. creates Pods for a Deployment)
6. **Scheduler** assigns unscheduled Pods to appropriate nodes
7. **kubelet** on the target node pulls the image and starts the container via the container runtime (containerd/CRI-O)

Quick Command:

```
# Apply and watch events in real time
kubectl apply -f deployment.yaml
kubectl get events --sort-by=.lastTimestamp -w
```


Q1
5

What is kube-proxy?

kube-proxy is a network proxy running on every node. It maintains **iptables / IPVS rules** that implement Kubernetes Service networking — translating a Service ClusterIP:Port to the actual Pod IPs.

Modes:

- **iptables** (default) – adds NAT rules; random Pod selection per connection
- **IPVS** – kernel-level load balancing; better performance at scale; more LB algorithms (rr, lc, wrr...)
- **userspace** (legacy) – deprecated, very slow

Example — how a Service call is routed:

```
# Service ClusterIP: 10.96.0.10:80
# Pod endpoints: 10.244.1.5:8080, 10.244.2.7:8080

# kube-proxy creates iptables rules like:
# -A KUBE-SVC-XXX -m statistic --mode random --probability 0.5 -j KUBE-SEP-AAA
# -A KUBE-SVC-XXX -j KUBE-SEP-BBB

# Check current proxy mode
kubectl get configmap kube-proxy -n kube-system -o yaml | grep mode
```

What is a Namespace?

A **Namespace** is a virtual cluster inside a Kubernetes cluster. It provides a scope for names, resource quotas, and access control — enabling multiple teams or environments to share the same physical cluster.

Default Namespaces:

- **default** – where objects go if no namespace is specified
- **kube-system** – Kubernetes core components (DNS, kube-proxy, etc.)
- **kube-public** – publicly readable data (e.g. cluster-info ConfigMap)
- **kube-node-lease** – Node heartbeat lease objects

Example:

```
# Create a namespace
kubectl create namespace staging

# Apply resources to a namespace
kubectl apply -f app.yaml -n staging

# Set default namespace for current context
kubectl config set-context --current --namespace=staging

# ResourceQuota to limit a namespace
apiVersion: v1
kind: ResourceQuota
metadata:
  name: staging-quota
  namespace: staging
spec:
  hard:
    requests.cpu: "4"
    requests.memory: 8Gi
    limits.cpu: "8"
    limits.memory: 16Gi
    pods: "20"
```

Q1
7

What causes Pods to remain in Pending state?

A Pod in **Pending** state means the scheduler cannot place it on any node yet.

Common Causes:

- **Insufficient resources** – no node has enough CPU/memory to satisfy the Pod's requests
- **PVC not bound** – the Pod requests a PersistentVolumeClaim that hasn't been provisioned
- **Node selector / affinity mismatch** – no nodes match the required labels
- **Taints & tolerations** – all nodes are tainted and the Pod has no matching toleration
- **Scheduler disabled / not running**
- **Image pull backoff** – sometimes stays Pending before becoming ErrImagePull

Debugging:

```
kubectl describe pod <pending-pod>
# Key section: "Events" – look for "FailedScheduling"
# Example event:
# 0/3 nodes are available:
#   1 Insufficient cpu.
#   2 node(s) had untolerated taint {node-role.kubernetes.io/control-plane}

# Check node resources
kubectl describe nodes | grep -A5 "Allocated resources"
```

Q1
8

What is a DaemonSet?

A **DaemonSet** ensures that a copy of a Pod runs on **every node** (or a subset of nodes) in the cluster. When a new node joins the cluster, the DaemonSet automatically schedules the Pod on it.

Common Use Cases:

- Log collectors (Fluentd, Filebeat)
- Monitoring agents (Prometheus Node Exporter, Datadog)
- Network plugins (Calico, Cilium)
- Storage daemons (Ceph, GlusterFS clients)

Example:

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: node-exporter
  namespace: monitoring
spec:
  selector:
    matchLabels:
      app: node-exporter
  template:
    metadata:
      labels:
        app: node-exporter
    spec:
      hostNetwork: true
      tolerations:
        - key: node-role.kubernetes.io/control-plane
          effect: NoSchedule          # run on control-plane too
      containers:
        - name: node-exporter
          image: prom/node-exporter:v1.7.0
          ports:
            - containerPort: 9100
              hostPort: 9100
```

Q1
9

What is a ReplicaSet?

A **ReplicaSet** ensures a specified number of Pod replicas are running at all times. If a Pod dies, the ReplicaSet creates a new one; if there are too many, it deletes the excess.

Deployment vs ReplicaSet: A Deployment *owns* a ReplicaSet and adds rolling-update and rollback capabilities on top. In practice, you rarely create a ReplicaSet directly — you use a Deployment.

Example:

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: my-rs
spec:
  replicas: 3
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
        - name: my-app
          image: nginx:1.25

# Scale imperatively
kubectl scale replicaset my-rs --replicas=5

# Check status
kubectl get rs my-rs
```

Tip: ReplicaSet uses label selectors — if you manually create a Pod with matching labels, the RS will count it towards the desired replicas.

Role-Based Access Control (RBAC) is Kubernetes' authorisation mechanism. It controls *who* can do *what* on *which* resources.

Core RBAC Objects:

- **Role** – defines permissions within a single namespace
- **ClusterRole** – defines permissions cluster-wide
- **RoleBinding** – grants a Role to a user/group/ServiceAccount in a namespace
- **ClusterRoleBinding** – grants a ClusterRole cluster-wide

Example — give a developer read-only access to Pods:

```
# 1. Create a Role in the 'dev' namespace
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: dev
  name: pod-reader
rules:
- apiGroups: [""]
  resources: ["pods", "pods/log"]
  verbs: ["get", "list", "watch"]

---
# 2. Bind it to a user
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: read-pods
  namespace: dev
subjects:
- kind: User
  name: alice
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io

# 3. Check permissions
kubectl auth can-i list pods --namespace=dev --as=alice
```